Elective course (1) NEURAL NETWORK AND DEEP LEARNING

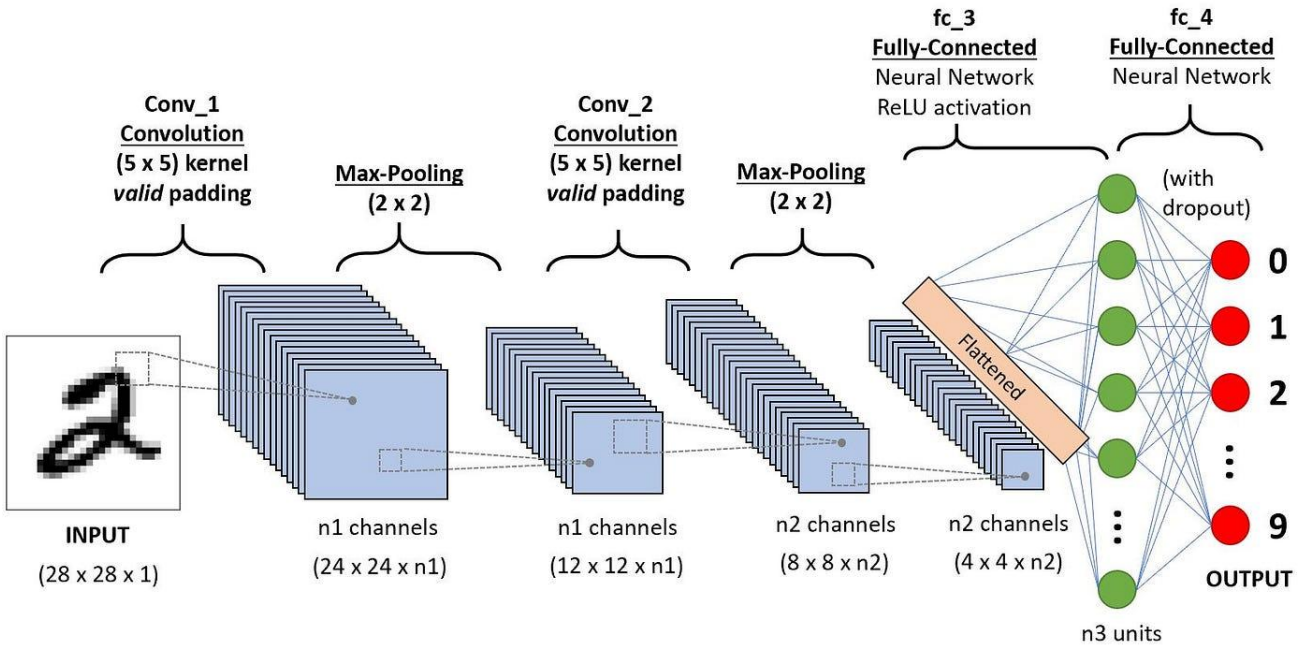
Third year computer science

chapter 5

Dr. Menas Ebrahim Eissa

Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are a class of deep neural networks specifically designed for processing structured grid-like data, such as images. CNNs are widely used in computer vision tasks like image classification, object detection, and segmentation.



Key Components of CNNs

1- Convolution Layer

2- Activation Function

3- Pooling Layer

4- Flatten Layer

5- Fully Connected Layer (FC Layer)

6- Softmax Layer (for Classification)

Key Components of CNNs

1- Convolution Layer

- **Uses filters (kernels) to extract spatial features from the input image.**
- **Each filter slides over the input image and performs element-wise multiplication and summation.**
- **The output is a feature map that highlights patterns like edges and textures.**

Types of Filters

1 Identity Filter

```python
[[ 0,  0,  0],
 [ 0,  1,  0],
 [ 0,  0,  0]]
```

Leaves the image unchanged; used to test the filtering pipeline itself.

Types of Filters

2 Edge Detection 1

python

```
[[-1, -1, -1],
 [-1,  8, -1],
 [-1, -1, -1]]
```

Detects strong edges (all directions); highlights boundaries and outlines.

Types of Filters

3 Edge Detection 2

python

```
[[ 1,  0, -1],
 [ 0,  0,  0],
 [-1,  0,  1]]
```

Detects diagonal edges; focuses on sharp diagonal transitions in brightness.

Types of Filters

4 Sharpen

```python
[[ 0, -1,  0],
 [-1,  5, -1],
 [ 0, -1,  0]]
```

Enhances fine details; makes edges and textures more pronounced.

Types of Filters

5 Box Blur

```python
[[1/9, 1/9, 1/9],
 [1/9, 1/9, 1/9],
 [1/9, 1/9, 1/9]]
```

Smooths the image uniformly; averages neighboring pixels to blur detail.

Types of Filters

6 Gaussian Blur

(A typical 3×3 Gaussian kernel)

python

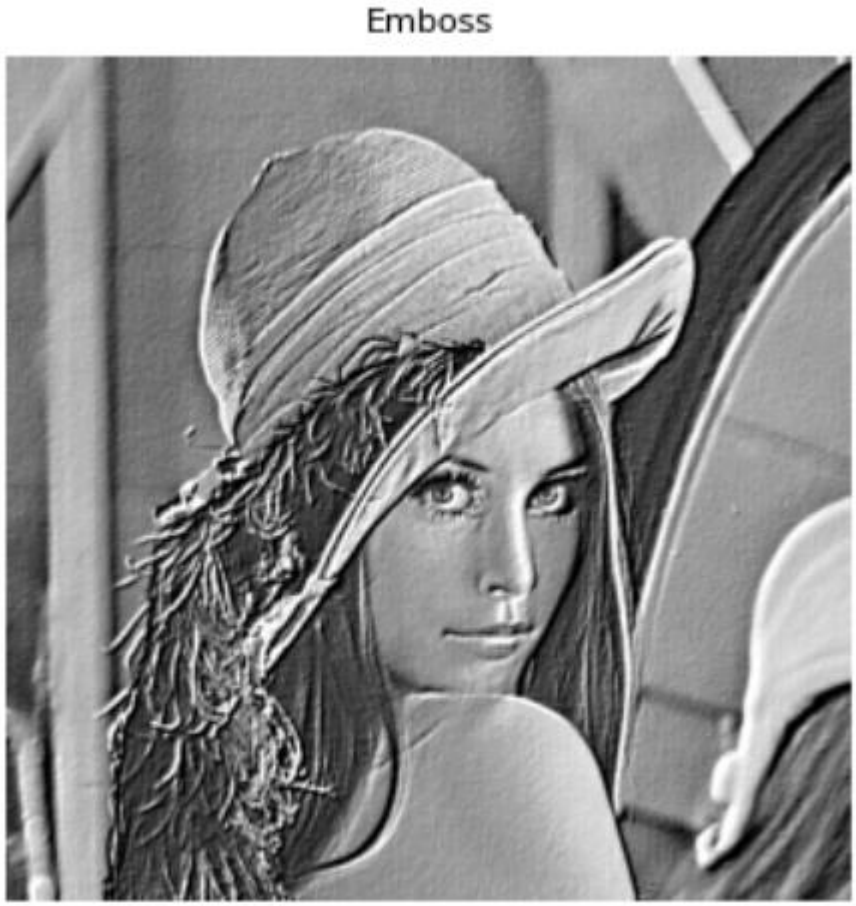
```
[[1/16, 2/16, 1/16],
 [2/16, 4/16, 2/16],
 [1/16, 2/16, 1/16]]
```

Softens the image more gently (weighted average); preserves overall shape.

Types of Filters

7 Emboss

```python
[[-2, -1,  0],
 [-1,  1,  1],
 [ 0,  1,  2]]
```



Creates a 3D "embossed" effect by highlighting edges in a single direction.

Types of Filters

8 Sobel X (Vertical Edges)

python

```
[[-1,  0,  1],
 [-2,  0,  2],
 [-1,  0,  1]]
```

- **Detects vertical edges; emphasizes changes from left to right.**

Types of Filters

9 Sobel Y (Horizontal Edges)

python

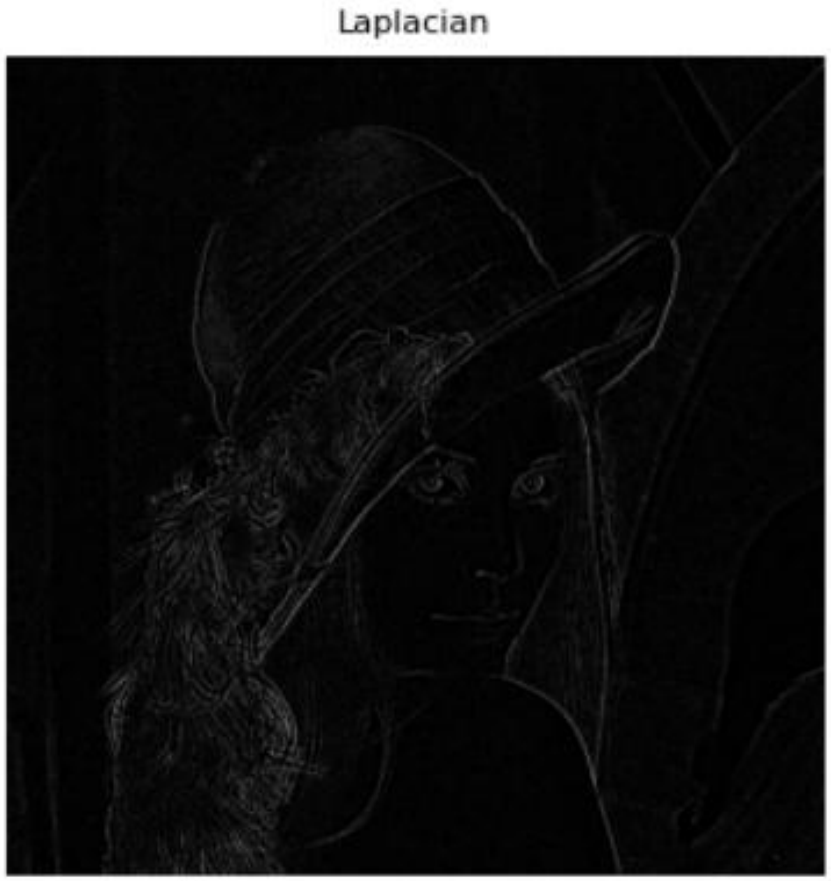
```
[[-1, -2, -1],
 [ 0,  0,  0],
 [ 1,  2,  1]]
```

Detects horizontal edges; emphasizes changes from top to bottom.

Types of Filters

10 Laplacian

```python
[[ 0,  1,  0],
 [ 1, -4,  1],
 [ 0,  1,  0]]
```



- **Detects edges in all directions; focuses on areas with rapid intensity change.**

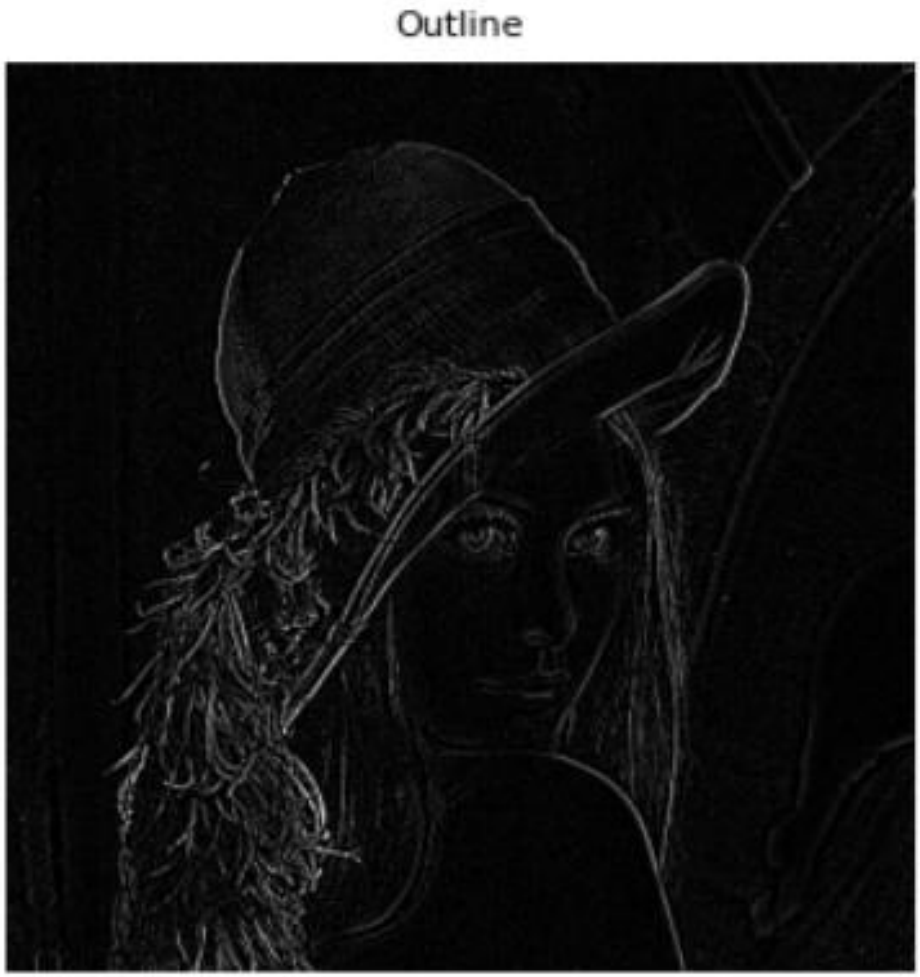
Types of Filters

1 1 Outline

(Same as Edge Detection 1; included for emphasis)

python

```
[[-1, -1, -1],
 [-1,  8, -1],
 [-1, -1, -1]]
```



Similar to Edge Detection 1; strongly highlights outlines and contours.

Types of Filters

1️⃣2️⃣ Top Sobel

```python
[[-1, -2, -1],
 [ 0,  0,  0],
 [ 1,  2,  1]]
```

Detects edges facing upwards; good for horizontal structures facing up.

Types of Filters

1 3 Bottom Sobel

```python
[[ 1,  2,  1],
 [ 0,  0,  0],
 [-1, -2, -1]]
```

Detects edges facing downwards; captures horizontal edges at the bottom side.

Types of Filters

14 Left Sobel

```python
[[ 1,  0, -1],
 [ 2,  0, -2],
 [ 1,  0, -1]]
```

Detects edges facing left; enhances vertical features on the left side.

Types of Filters

1 5 Right Sobel

python

```
[[-1,  0,  1],
 [-2,  0,  2],
 [-1,  0,  1]]
```

Detects edges facing right; highlights vertical edges on the right side.

Key Components of CNNs

2- Activation Function

- Commonly used activation functions include ReLU (Rectified Linear Unit), which introduces non-linearity to help the network learn complex features.

💡 Why is this important?

- Many filters (like edge detectors) produce **negative and positive outputs.**
- Without non-linearity:
  - Stacking layers would just **combine filters linearly.**
- With non-linearity:
  - The network can **build hierarchical, complex representations.**

Key Components of CNNs

3- Pooling Layer

- **Reduces the spatial dimensions of feature maps while retaining important information.**
- **Types:**
  - **Max Pooling: Takes the maximum value in a region.**
  - **Average Pooling: Takes the average value in a region.**

🔍 Example: Max Pooling

🖼️ Suppose you have a 4×4 feature map:

lua Copy Edit

```
[[1, 3, 2, 4],
 [5, 6, 7, 8],
 [9, 2, 1, 5],
 [4, 3, 0, 6]]
```

🔨 Apply 2×2 Max Pooling (stride = 2):

You divide the feature map into **non-overlapping 2×2 regions:**

1️⃣ Region 1:

lua

```
[[1, 3],
 [5, 6]]
```

➡️ Max = 6

2️⃣ Region 2:

lua

```
[[2, 4],
 [7, 8]]
```

➡️ Max = 8

3️⃣ Region 3:

lua

```
[[9, 2],
 [4, 3]]
```

➡️ Max = 9

4️⃣ Region 4:

lua

```
[[1, 5],
 [0, 6]]
```

➡️ Max = 6

🟢 Result after pooling:

```lua
[[6, 8],
 [9, 6]]
```

✅ The feature map shrank from 4×4 to 2×2!

Key Components of CNNs

4- Flatten Layer

- **Purpose: Flattening** converts the 2D or 3D output from the previous layers into a **1D vector** so it can be fed into fully connected (dense) layers.
- **What happens here:** After the convolution and pooling layers, the feature maps are **flattened** into a 1D array to serve as input for dense layers

Key Components of CNNs

5- Fully Connected Layer (FC Layer)

- **Flattens the feature maps into a vector.**
- **Uses dense layers to classify the image based on extracted features.**

Key Components of CNNs

6- Softmax Layer (for Classification)

- **Converts the final outputs into probabilities for multi-class classification.**

Important notes

🚶 What is stride in a convolution?

👉 **Stride** controls **how far the filter moves** across the input image.

- **Stride = 1:**

  The filter slides **one pixel at a time** (left → right, top → bottom).

- **Stride = 2:**

  The filter **jumps two pixels** at a time. This **skips over some positions**, which **shrinks the output**.

🔍 Example: Visualizing stride

Let's say you have a **5×5 input image** and a **3×3 filter.**

🟢 Stride = 1:

pgsql Copy Edit

```
Step 1: Apply filter to (top-left corner)
Step 2: Move filter 1 pixel right
Step 3: Move filter 1 pixel right (until end of row)
Then move 1 row down and repeat.
```

🟠 Stride = 2:

pgsql Copy Edit

```
Step 1: Apply filter to (top-left corner)
Step 2: Move filter 2 pixels right (skipping middle)
Step 3: Move filter 2 pixels down to next row.
```

📦 What is padding in CNNs?

Padding means **adding extra pixels** (usually zeros) **around the border of the input image** before applying the convolution.

👉 The main purpose is to **control the spatial size of the output.**

👀 Why use padding?

Without padding, every convolution **shrinks the image.**

1️⃣ 'Valid' padding:

- Means: **no padding**
- Shrinks output
- Formula:

$$W_{\text{out}} = \frac{W_{\text{in}} - \text{Kernel Size}}{\text{Stride}} + 1$$

2️⃣ 'Same' padding:

- Adds just enough padding to **keep the output size the same** as the input (if stride = 1).

🛠️ 1️⃣ What causes the image to downsize in CNNs?

The **main things that reduce image size** are:

- **Valid Convolutions** (no padding)
- **Strides > 1**
- **Pooling Layers** (e.g., Max Pooling)

🔍 2️⃣ Basic convolution formula

The **output size (W_out, H_out)** of a convolution layer is:

$$W_{\text{out}} = \frac{W_{\text{in}} - \text{Kernel Size} + 2 \times \text{Padding}}{\text{Stride}} + 1$$

$$H_{\text{out}} = \frac{H_{\text{in}} - \text{Kernel Size} + 2 \times \text{Padding}}{\text{Stride}} + 1$$

📦 Example: 32×32 input, 3×3 kernel, stride 1, padding 0 ("valid" convolution)

Plug into the formula:

$$W_{\text{out}} = \frac{32 - 3 + 0}{1} + 1 = 30$$

So a **32×32 input becomes 30×30** after one convolution.

🧐 What if we use padding?

With **same padding** (pad = 1):

$$W_{\text{out}} = \frac{32 - 3 + 2(1)}{1} + 1 = 32$$

✅ **No downsize!** (This is common in modern CNNs.)

🌀 3 Stride effect

If you **increase stride** (e.g., stride = 2), downsizing is more dramatic.

- 32×32 input
- 3×3 kernel
- stride = 2
- pad = 0

$$W_{\text{out}} = \frac{32 - 3 + 0}{2} + 1 = 15.5 \rightarrow 15$$

👀 We usually **floor** to the nearest integer → 15×15 output.

🌊 4️⃣ Pooling layers

MaxPooling (2×2 pool, stride 2):

- **Input: 32×32**
- Pool 2×2 (stride 2)

This **halves** each dimension:

➡️ **Output: 16×16**

Pooling is a **common reason** CNNs downsize images!

🔗 5️⃣ Stacking layers: How it adds up

Example architecture:

Layer	Size change
Input: 32×32×3	32×32
Conv (3×3, stride 1, same pad)	32×32
MaxPool (2×2, stride 2)	16×16
Conv (3×3, stride 1, same pad)	16×16
MaxPool (2×2, stride 2)	8×8
Conv (3×3, stride 1, valid)	6×6
…	…

✅ So to summarize:

- **Padding** = **'valid'** → shrinks the image.
- **Stride** > **1** → shrinks faster.
- **Pooling layers** → aggressively reduce size (often by 2×).

In practice, CNNs **compress spatial size** but **increase depth** (channels), letting them capture **abstract patterns** at different scales.

Important topic
Do I choose which filter to be used??

- The code does not pre-choose any filters (like an edge detector or blur filter).
- Instead, it starts with random filters and learns the best filters automatically during training!

🚦 Step-by-step: What happens under the hood?

1️⃣ At the start:

- When you write:

```python
layers.Conv2D(32, (3, 3), ...)
```

  TensorFlow **creates 32 filters, each 3×3×channels (e.g., 3×3×1 if grayscale).**

- These filters are **randomly initialized.**

2️⃣ During training:

- The CNN processes images, computes the loss (how wrong it is), and **adjusts the filters** using **backpropagation + gradient descent.**
- For example:
  - If one filter starts to detect **vertical edges** and that helps improve accuracy, the training process **strengthens that filter.**
  - If a filter is useless, its weights may shrink toward zero or adapt into something better.

3️⃣ **Result:**

- Over time, the CNN **learns filters that specialize:**
  - Some may detect **edges**
  - Others may detect **curves, textures, patterns**
  - In deeper layers, filters might recognize **complex shapes** (like digits or faces).

Applications of CNN

CNNs automate feature extraction, making them powerful tools for analyzing visual data. Applications include:

- **Image Classification**: Identifying objects in an image (e.g., cats vs. dogs).
- **Object Detection**: Locating objects within an image.
- **Facial Recognition**: Identifying people based on facial features.
- **Medical Image Analysis**: Detecting diseases in medical scans.